The Australian National University

School of Computing

# JHQ-GPU: Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

DRAFT TEMPLATE — WRITING IN PROGRESS

# Declaration and Contributions

**TO WRITE:** Replace this page with the declaration required by the current Canvas instructions. State the work you personally performed, identify inherited JHQ algorithms and code, and distinguish your contributions from those of the supervisor, collaborators, and generative-AI tools. Do not sign an unverified claim of sole authorship or prior non-submission.

## Research ownership

**TO WRITE:** Describe your contribution to system design, implementation, experimental design, analysis, and writing. Cite the original JHQ paper and identify the relationship to the coauthored ADC manuscript.

## Use of generative-AI tools

**TO WRITE:** Complete the record in Appendix C. Record actual tools, purposes, relevant prompts or prompt-log locations, and your review of outputs. Match the current course disclosure instructions.

## Signature and date

**TO WRITE:** Add a signature and date only if the course requires them and after checking the final declaration wording.

# Abstract

**TO WRITE:** Write this after the results and conclusion. A useful planning length is 250–350 words, subject to Canvas instructions. Cover the problem, your own methods, the strongest verified results, and the principal quality or deployment limitation. Keep experimental versions consistent. Do not describe this template or the writing blueprint in the final abstract.

# Acknowledgements

**TO WRITE:** Acknowledge actual supervision, collaboration, computing resources, and other assistance. This page is optional unless the course specifies it; remove its input from main.tex if unused.

# Contents

**Declaration and Contributions** i

**Abstract** ii

**Acknowledgements** iii

**Notation and Abbreviations** xi

**1 Introduction** 1

1.1 Context and Practical Objectives . . . . . 1
1.2 Problem Definition and Scope . . . . . 1
1.3 Three Linked Technical Challenges . . . . . 1
1.4 Research Questions and Falsifiable Claims . . . . . 1
1.5 Contributions and Research Ownership . . . . . 1
1.6 Thesis Roadmap . . . . . 2

**2 Background and Critical Related Work** 3

2.1 Formal ANN Objective and Metrics . . . . . 3
2.2 IVF, PQ, ADC and Residual Refinement . . . . . 3
2.3 Inherited JHQ: Rotation, Cartesian Codes and Assumptions . . . . . 3
2.4 GPU Execution Concepts Used Here . . . . . 3
2.5 GPU Graph Search: SONG, CAGRA and BANG . . . . . 4
2.6 Quantised ANN, Fast Tables and Modern Alternatives . . . . . 4
2.7 Critical Synthesis and Precise Research Gap . . . . . 4

**3 Research Design and Measurement Methodology** 5

3.1 Research Questions as Controlled Comparisons . . . . . 5
3.2 Datasets, Queries, Ground Truth and Splits . . . . . 5
3.3 Measurement Boundary, Variance and Memory Accounting . . . . . 5
3.4 Baselines, Tuning Grids and Frontier Construction . . . . . 5
3.5 Versioned Claim-to-Evidence Ledger . . . . . 6
3.6 Evidence Strength and Threat Controls . . . . . 6

**4 System Architecture and GPU Index Construction** **7**
4.1 End-to-End Build and Search Boundaries . . . . . 7
4.2 Training, Rotation and Codebook Generation . . . . . 7
4.3 IVF Assignment, IDs and Physical Memory Layout . . . . . 7
4.4 Resident Storage, Workspace and Index Lifecycle . . . . . 7
4.5 Construction Cost and Tested Scale . . . . . 8
4.6 Index Representation and Safety Contracts . . . . . 8

**5 Representation-Aware GPU Search Execution** **9**
5.1 Query Path and Four Distinct Ranking Objects . . . . . 9
5.2 Exact Cartesian LUT Factorisation . . . . . 9
5.3 Packed Codes and Candidate-Parallel Memory Access . . . . . 9
5.4 Exact Global Primary Top-c via Multi-Block Scanning . . . . . 9
5.5 Shared Complete-Key Thresholds and Execution Scheduling . . . . . 10
5.6 Lane-Cooperative Residual Scoring and Final Top-k . . . . . 10
5.7 IVF Probe Selection as an Independent Primitive . . . . . 10

**6 Workload-Level Refinement-Budget Calibration** **11**
6.1 Why Fixed Refinement Budgets Can Be Wasteful . . . . . 11
6.2 GT-Free Sampled Output Agreement . . . . . 11
6.3 Budget Search, Nestedness and Tie Safety . . . . . 11
6.4 Budget Baselines and Their Questions . . . . . 11
6.5 Hold-out Risk, Calibration Cost and Payback . . . . . 12
6.6 Policy Failure Modes and Applicability . . . . . 12

**7 Experimental Evaluation and Research Findings** **13**
7.1 Final Experiment Manifest and Comparison Families . . . . . 13
7.2 Modern GPU ANN Recall–QPS Frontiers . . . . . 15
7.3 Cartesian LUT, Packed Loads and Lane Refinement . . . . . 18
7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation . . . . 21
7.5 Held-out Budget Quality, Throughput and Payback . . . . . 24
7.6 Batch Regimes, G Scheduling and Workload Skew . . . . . 29
7.7 Cold Construction, Resident Memory and Tested Scale . . . . . 32
7.8 Numerical Robustness, Anomalies and Recall Attribution . . . . . 35

**8 Discussion, Limitations and Future Research** **38**
8.1 Three Cross-Layer Insights . . . . . 38

8.2 Two Deep Negative-Result Case Studies . . . . . 38
8.3 Threats to Internal and External Validity . . . . . 38
8.4 Original Testable Research Directions . . . . . 38
8.5 Conditional Deployment Recommendation . . . . . 38

**9 Conclusion** **40**
9.1 Answers to the Four Research Questions . . . . . 40
9.2 Closing Principle and Outlook . . . . . 40

**Bibliography** **41**

**A Reproducibility and Experiment Manifest** **42**
A.1 Source and Build Environment . . . . . 42
A.2 Data and Trained States . . . . . 42
A.3 Result-to-Figure Mapping . . . . . 42

**B Supplementary Results and Correctness Checks** **43**
B.1 Complete Parameter Grids . . . . . 43
B.2 Correctness and Numerical Checks . . . . . 43
B.3 Additional Results and Negative Designs . . . . . 43

**C Contributions and Tool-Use Record** **44**
C.1 Contribution Record . . . . . 44
C.2 Generative-AI Assistance . . . . . 44

**D Template Examples — Draft Only** **45**
D.1 Citations and Cross-References . . . . . 45
D.2 A Source-Native Diagram . . . . . 45
D.3 Tables . . . . . 45
D.4 Equations and Proofs . . . . . 45
D.5 Pseudocode and Code . . . . . 46

# List of Figures

7.1 Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.85; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis. . . . . 16

7.2 Completed v71 Cartesian table-grouping experiment, retained from the existing figure set. Group counts describe $g_{\mathrm{LUT}}$, not parallel scan groups. Curves use median QPS normalized to the one-group arm. . . 19

7.3 Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. These panels come from different experiment families. . . . . 20

7.4 Existing v69 held-out risk figure, checked against the budget log. The vertical axis counts calibration draws whose held-out recall loss exceeds $10^{-3}$. Increasing $S$ also tightens the allowed slot disagreement fraction because the threshold in equation (7.1) remains one. . . . . 26

7.5 Existing v69 payback figure. Payback is an optimistic estimate using mean selected-arm QPS and 500-query equivalents. ArXiv has no throughput payback. Table 7.7 retains actual held-out counts. . . . . 28

7.6 Batch-dependent execution. Left: integrated shared-bound fixed group counts and scheduled execution relative to v61 L16; the full-batch population has 11 cells, versus 13 at smaller batches. Right: v62 batch-8 profiled scan time; merge time is additional. The panels separate application throughput from the stage evidence explaining small-batch gains. . . . . 30

7.7 Recorded construction costs, with CAGRA int8 included. JHQ is one cold train-plus-add record; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic. . . . . 33
7.8 Completed resident-memory decomposition, verified against the v71 two-group measurements. Component sizes are modelled, markers are measured used totals, and hatching retains the unattributed remainder. This is neither a construction-peak trace nor an index-only measurement. 34
7.9 Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE uses 1,000 queries and Stella 999. An endpoint is the end of the tested grid, not an intrinsic recall limit. . . 35

D.1 Formatting example: a schematic query path. Replace or extend it with the verified execution boundaries of the implementation. . . . . 45

# List of Tables


7.1 Evidence families. Counts denote measured configurations or checks, not independent index builds. All use the RTX 5090; timing protocols differ. . . . . 14

7.2 Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator. . 17

7.3 Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged. . . . . 20

7.4 v69/v65 end-to-end throughput ratios for the exact FP32 probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Values above one favour v69. . . . . 23

7.5 Held-out quality at $S = 128$. Rule $\alpha$ is the median across 64 draws; oracle $\alpha$ uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; $L_{95}$ is the logged lower-order 95th-percentile loss estimate. $R_{100}$ is fixed-budget recall. . . . . 25

7.6 Universal fixed-budget controls over the twelve held-out cells. Qualifying means within $10^{-3}$ of the best tested recall, using printed precision. QPS penalty is the largest fractional deficit relative to the cell's quality-constrained oracle, irrespective of whether that fixed arm itself qualifies. . . . . 27

7.7 v69 calibration economics, $S = 128$. Rates are kQPS; rule rate is mean selected-arm QPS. Payback uses the actual held-out pool size (500, or 499 for Stella). "Never" means estimated selected execution is slower than fixed $\alpha = 100$, so throughput cannot repay calibration. 28

7.8 Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. "Held out" excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment's held-out query split. . . . 30

7.9 Cold JHQ construction and separately measured resident memory. Train/add come from one recorded cold build per dataset. Memory comes from v71's two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. "Residual" is the modelled residual-code share of the measured used total, not the fraction read per query. . . . . 32

D.1 Example manifest fields. This table contains no experimental results. 46

D.2 Pseudocode layout example. . . . . 46

# Notation and Abbreviations

| Symbol | Meaning |
|---|---|
| $N, d$ | Number of database vectors and original dimension |
| $M, D_s$ | Number of primary subspaces and dimensions per subspace |
| $k$ | Number of returned neighbours |
| $n_c, c$ | Routed candidates and primary survivors receiving refinement |
| $\alpha$ | Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$ |
| $S, \epsilon$ | Calibration sample size and allowed disagreeing result slots |
| $g_{\text{LUT}}$ | Number of groups in a factorised lookup table |
| $g_{\text{scan}}$ | Number of scan groups per query |
| $\ell_{\text{ref}}$ | Number of cooperating lanes per refined candidate |
| $D_{\text{raw}}$ | Squared distance between original vectors |
| $D_{\text{P}}$ | Distance under the primary reconstruction |
| $D_{\text{H}}$ | Distance under the hierarchical reconstruction |

| Abbreviation | Meaning |
|---|---|
| ANN | Approximate nearest neighbour |
| ADC | Asymmetric distance computation (distinct from the ADC conference) |
| IVF | Inverted file |
| JHQ | Johnson–Lindenstrauss enhanced hierarchical quantisation |
| LUT | Lookup table |
| PQ | Product quantisation |
| QPS | Queries per second |
| GT | Ground truth |

**TO WRITE:** Check every definition against the final implementation and remove symbols that are not used. Keep code parameter names distinct from mathematical notation, especially the multiple uses of G and B.

Chapter 1

# Introduction

**TO WRITE:** Establish the research problem, testable questions, scope, and personal contributions. Keep results consistent with the final evidence manifest.

## 1.1 Context and Practical Objectives

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 1.2 Problem Definition and Scope

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 1.3 Three Linked Technical Challenges

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 1.4 Research Questions and Falsifiable Claims

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 1.5 Contributions and Research Ownership

**TO WRITE:** Identify personal work, collaborator roles, inherited algorithms and code, the ADC manuscript relationship, and actual tool assistance.

## 1.6 Thesis Roadmap

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 2

# Background and Critical Related Work

**TO WRITE:** Explain only background used by the thesis. Compare alternatives critically and distinguish inherited JHQ properties from this work.

## 2.1 Formal ANN Objective and Metrics

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 2.2 IVF, PQ, ADC and Residual Refinement

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 2.3 Inherited JHQ: Rotation, Cartesian Codes and Assumptions

**TO WRITE:** State that the orthogonal transform is square. Define the primary codebook using its actual formula, residual relative to primary reconstruction, and valid parameters.

## 2.4 GPU Execution Concepts Used Here

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 2.5 GPU Graph Search: SONG, CAGRA and BANG

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 2.6 Quantised ANN, Fast Tables and Modern Alternatives

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 2.7 Critical Synthesis and Precise Research Gap

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 3

# Research Design and Measurement Methodology

**TO WRITE:** Define comparison protocols once. Separate exploratory findings from questions specified before measurement; identify the actual evidence for each claim.

## 3.1 Research Questions as Controlled Comparisons

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 3.2 Datasets, Queries, Ground Truth and Splits

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 3.3 Measurement Boundary, Variance and Memory Accounting

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 3.4 Baselines, Tuning Grids and Frontier Construction

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 3.5 Versioned Claim-to-Evidence Ledger

**TO WRITE:** Record commit, binary, compiler options, index/cache identity, query split, raw result, metric, and figure-generation script for every main claim.

## 3.6 Evidence Strength and Threat Controls

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 4

# System Architecture and GPU Index Construction

**TO WRITE:** Describe build-time dataflow, CPU/GPU responsibilities, physical storage, temporary memory, and representation invariants.

## 4.1 End-to-End Build and Search Boundaries

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 4.2 Training, Rotation and Codebook Generation

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 4.3 IVF Assignment, IDs and Physical Memory Layout

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 4.4 Resident Storage, Workspace and Index Lifecycle

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 4.5 Construction Cost and Tested Scale

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 4.6 Index Representation and Safety Contracts

**TO WRITE:** Distinguish representation checks, algorithmic selection correctness, numerical agreement, and memory-safety checks; do not use recall equality as proof.

Chapter 5

# Representation-Aware GPU Search Execution

**TO WRITE:** For each design, explain the original cost, alternatives, actual mechanism, correctness premises, resource cost, and matching controlled experiment.

## 5.1 Query Path and Four Distinct Ranking Objects

**TO WRITE:** Distinguish raw-vector, primary-score, refined-score, and deterministic ranking-key orderings. State what each stage preserves.

## 5.2 Exact Cartesian LUT Factorisation

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 5.3 Packed Codes and Candidate-Parallel Memory Access

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 5.4 Exact Global Primary Top-c via Multi-Block Scanning

**TO WRITE:** Describe candidate partitioning, exact local selection, coverage of global top-c by the local union, and merge correctness under the actual stable-position tie rule.

## 5.5 Shared Complete-Key Thresholds and Execution Scheduling

**TO WRITE:** Explain complete-score threshold publication, its safety invariant, and scheduling. Do not describe the shared bound as partial-distance early exit.

## 5.6 Lane-Cooperative Residual Scoring and Final Top-k

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 5.7 IVF Probe Selection as an Independent Primitive

**TO WRITE:** Separate selection of IVF probes from primary candidate selection. Associate each reference test with the primitive it actually checks.

Chapter 6

# Workload-Level Refinement-Budget Calibration

**TO WRITE:** Define the policy, its reference outputs and assumptions, then separate sampled agreement, held-out quality, and calibration economics.

## 6.1 Why Fixed Refinement Budgets Can Be Wasteful

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 6.2 GT-Free Sampled Output Agreement

**TO WRITE:** Define the disagreement-slot sum and distinguish it from a per-query rate. Explain how fixed epsilon changes strictness as the sample size changes.

## 6.3 Budget Search, Nestedness and Tie Safety

**TO WRITE:** Check nested survivor sets, fixed refined scores, exact-selection configuration, deterministic ties, and the executed path before invoking monotonicity.

## 6.4 Budget Baselines and Their Questions

**TO WRITE:** Explain the distinct questions answered by fixed budgets, same-sample enumeration, full-pool calibration, and the ground-truth oracle.

## 6.5 Hold-out Risk, Calibration Cost and Payback

**TO WRITE:** Separate selected-arm throughput from total cost. State held-out quality loss, sampling variability, calibration overhead, and reuse assumptions.

## 6.6 Policy Failure Modes and Applicability

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 7

# Experimental Evaluation and Research Findings

The evaluation asks whether the representation and execution choices in JHQ-GPU improve useful ANN operating points, whether parallel execution preserves the intended candidate selection, and whether a workload can reduce refinement effort without an unacceptable loss of quality. These questions require different evidence. A faster kernel does not establish a better recall–throughput frontier, agreement with a reference result does not establish ground-truth recall, and a resident allocation does not measure construction peak memory. The experiments below keep these distinctions explicit.

The main findings are conditional but consistent. Two-group Cartesian tables and packed access improve screening; cooperative refinement reduces residual-scoring time; and candidate-parallel scanning is especially useful when the query batch supplies too few blocks. At batch 1,000, JHQ-GPU offers competitive operating regions against modern GPU baselines, with both low-recall and high-recall exceptions. Budget calibration often improves the estimated throughput of subsequent queries, but its held-out quality varies across workloads. The results support a system with several useful execution regimes rather than a single configuration that dominates all alternatives.

## 7.1 Final Experiment Manifest and Comparison Families

### 7.1.1 Evidence families and measurement boundaries

Table 7.1 identifies the experiment families used in this chapter. Version numbers name experimental implementations, not successive measurements of one unchanged benchmark. In particular, the historical ADC paper and the later v69/v71 runs have different timing results. The newer raw records determine the corresponding numbers here; historical packing and refinement experiments remain separate ablations.

**Table 7.1:** Evidence families. Counts denote measured configurations or checks, not independent index builds. All use the RTX 5090; timing protocols differ.

| Family | Purpose and implementation | Coverage |
|---|---|---|
| E1 | v69 versus cuVS graph, PQ, RaBitQ and brute force | 1,590 points; 4 datasets |
| LUT | v71 table grouping, $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ | 24 cells; 96 arms |
| Packing | Historical byte/word access factorial | 48 paired cells |
| Refinement | v61 scalar/load/lane ablation | 13 configurations |
| Scan/bound | v62 global selection; v63 shared bound | 64 / 48 correctness checks |
| Scheduling | Integrated v65 execution policy | 50 workload–batch cells |
| Probe | v69 exact FP32 probe selection | 1,792 synthetic cases |
| Budget | v69 sampled, full-pool and oracle budgets | 12 cells; 64 draws per $S$ |
| Scale | v69 search, cold builds and resident memory | BGE and Stella extensions |

Multiplying their speedups would combine different baselines and would not yield a measured system speedup.

The E1 environment records an NVIDIA GeForce RTX 5090 with 32,607 MiB reported device memory, driver 595.71.05, CUDA 13.0 and cuVS 26.08.01. The container has a 25-CPU quota; OpenMP is limited to 16 threads. JHQ uses Release code compiled to compute_80 PTX and JIT-compiled for the device; the baselines use the installed native cuVS libraries. This is an implementation comparison on one device, not an architecture-independent algorithm ranking.

The principal cross-system timing boundary starts with FP32 queries in host memory and ends after top-10 identifiers and distances return to host memory and the stream synchronizes. It includes transfers, query preprocessing and search; CAGRA int8 query quantization is inside this boundary. Baselines reuse resource handles and output buffers. E1 uses five timing rounds, normally 24 calls per round, with fewer calls recorded for very slow baseline points. Its reported QPS is the median of round throughputs, each computed from the total queries and total elapsed time. It is not the reciprocal of the median individual-call latency. Warm-up, index construction and any budget calibration are excluded. Device-only baseline times are auxiliary: JHQ has no equivalent device-query API, so mixing those times into the main ratios would change the comparison.

### 7.1.2 Metrics and reproducibility

All experiments use squared L2 and $k = 10$. Recall is the mean fraction of ground-truth top-10 identifiers recovered, using only the first ten ground-truth entries even

when a file stores a wider row. The refinement factor determines $c = \max\{k, \lceil \alpha k \rceil\}$ survivors. Throughout this chapter $g_{\mathrm{LUT}}$ denotes Cartesian table grouping and $g_{\mathrm{scan}}$ denotes parallel scan groups; the two parameters perform different work.

The chapter's analysis script reads existing measurements without running new GPU experiments. It verifies record counts and correctness fields, recomputes paired ratios and matched-recall values, and writes the numeric tables directly from those records. A companion source ledger records SHA-256 hashes of consumed files. Its source-checkout identifier records the audited repository snapshot; it does not imply that every historical binary was built from that commit. Raw run headers and family-specific reports retain the individual build provenance. This separation makes the current figures reproducible without relabelling historical runs.[1]

## 7.2 Modern GPU ANN Recall–QPS Frontiers

### 7.2.1 Comparison scope and frontier construction

Figure 7.1 compares JHQ-GPU with CAGRA fp32, scalar-quantized CAGRA int8, IVF-RaBitQ, IVF-PQ and GPU brute force. CAGRA represents graph search; PQ and RaBitQ represent compressed inverted-file alternatives [2–4]. The four datasets are Vogue (932,328 × 768), arXiv (2,253,000 × 768), and the 999,000-vector OpenAI datasets at 1,536 and 3,072 dimensions. Each uses 1,000 queries and the same ground-truth file across methods.

Each method contributes its non-dominated measured operating points. An operating point is removed if another configuration of that method achieves at least its recall and throughput, with a strict improvement in one. Thus a frontier combines search parameters and sometimes distinct index configurations; it is not the trajectory of one fixed index. JHQ contributes 27–39 points per dataset, compared with 54–264 for the baseline families. The E1 parser finds 1,590 valid points, no failure records and no invalid-identifier or duplicate-row flags. This count establishes the size of the search sweep, not 1,590 independent replications.

The parameter coverage is substantive. CAGRA varies graph degree 32/64, search width 1–8 and intermediate top-$k$ capacity 32–512. RaBitQ varies 4/8/9-bit budgets, supported search modes and probe counts up to 1,024. PQ varies code length,

[1] The report repository provides `scripts/build_ch7.py`, `data/ch7/audit.json`, `data/ch7/matched.csv` and the source ledger `data/ch7/sources.csv`. The script requires the sibling `JHQ_GPU` checkout containing the archived measurements.

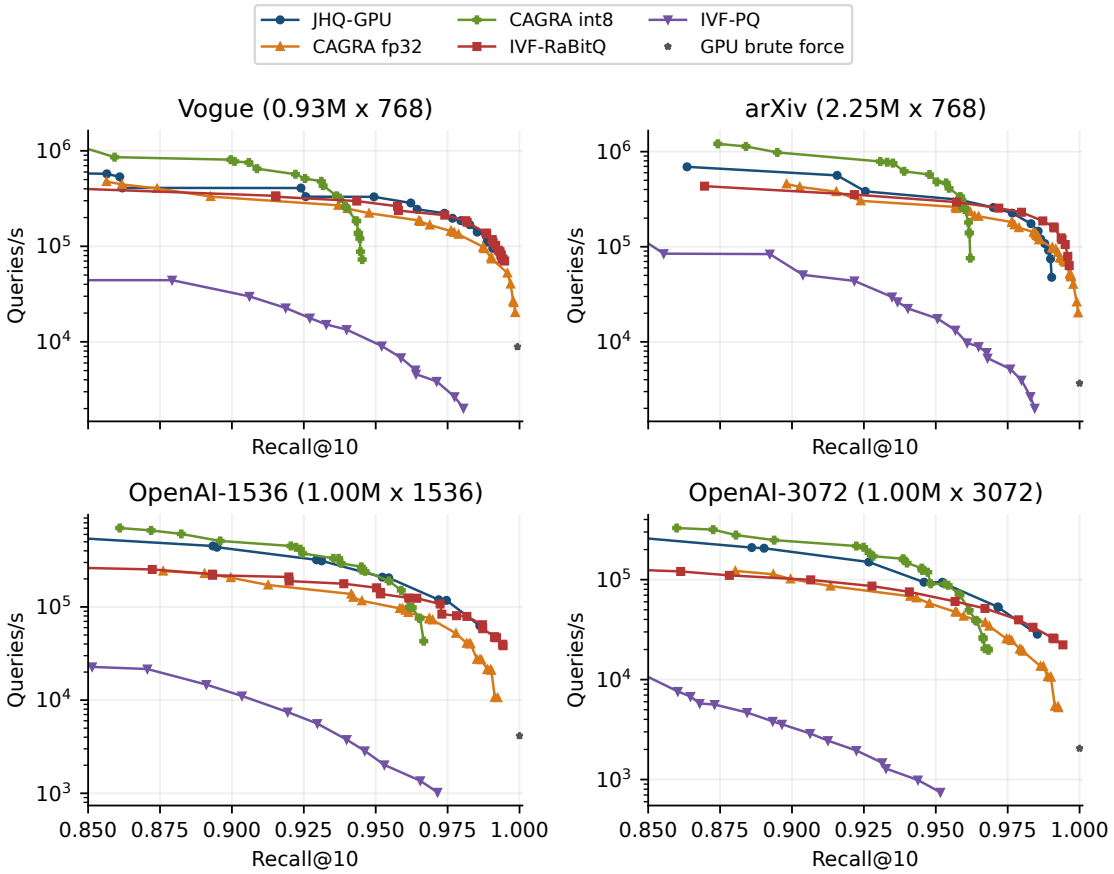



**Figure 7.1:** Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.85; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis.

lookup precision and probe count. JHQ varies both probe count and refinement budget, extending its initial grid where recall coverage is insufficient. The resulting comparison is conditional on these recorded grids. A curve ending before a target recall indicates an unmeasured operating region, not a proof that the method cannot reach it.

These curves compare achievable throughput, not equal memory consumption. JHQ stores approximately $d/8$ primary bytes and $d$ residual bytes per vector, plus metadata. RaBitQ at nine bits has a comparable nominal $9d/8$ code budget, but its 4/8-bit configurations use less storage. CAGRA int8 also stores graph edges; PQ code lengths vary; CAGRA fp32 retains FP32 vectors. Taking the envelope across these choices is useful for deployment decisions, but cannot isolate a representation advantage at a fixed memory budget. The memory evidence is discussed separately in Section 7.7.

**Table 7.2:** Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator.

| Dataset | JHQ kQPS | vs CAGRA fp32 | vs IVF-RaBitQ |
|---|---|---|---|
| Vogue | 230.8 | 1.416 / 1.537 | 1.059 / 1.046 |
| arXiv | 258.6 | 1.317 / 1.422 | 0.996 / 1.015 |
| OpenAI-1536 | 126.7 | 1.747 / 2.294 | 1.130 / 1.111 |
| OpenAI-3072 | 55.9 | 1.747 / 2.088 | 1.157 / 1.346 |

### 7.2.2 Matched recall and opposing operating regions

Table 7.2 quantifies the region around recall 0.97 using two estimators. The interpolated estimator is linear in log throughput between adjacent frontier points and never extrapolates beyond coverage. The measured estimator selects the fastest sampled point reaching the target. The interpolated estimator treats a measured recall within $5 \times 10^{-4}$ of the target as a match; the measured estimator requires recall at least equal to the target. Interpolation reduces sensitivity to unequal grid spacing; the measured estimator corresponds to an actually recorded configuration. Neither creates a new measurement at exactly the requested recall.

Against CAGRA fp32, interpolated ratios at this target range from 1.317× on arXiv to 1.747× on the two OpenAI datasets. Against IVF-RaBitQ, the corresponding values are 1.059×, 0.996×, 1.130× and 1.157×. The arXiv result is a near tie: its interpolated rate is slightly lower, while the measured threshold comparison is slightly higher. Reporting this as an unequivocal win would give the interpolation convention more evidential weight than the measurement supports.

Across targets 0.90–0.985, JHQ's interpolated advantage over CAGRA fp32 is approximately 1.10–2.44× in the covered comparisons. However, adding CAGRA int8 changes the low-recall interpretation: around 0.90–0.92, its lower-precision graph search can be faster than JHQ. At higher recall, IVF-RaBitQ often regains the lead, including several targets at 0.98 or above. On arXiv around 0.99, CAGRA fp32 is also faster than JHQ in the recorded grid. The defensible claim is therefore that JHQ has useful competitive regions, particularly around moderately high recall, rather than that it dominates modern GPU ANN methods.

The two estimators can disagree substantially where the grid is sparse. For example, at recall 0.97 the OpenAI-1536 comparison with CAGRA fp32 is 1.747× by interpolation but 2.294× using qualifying measured points. The latter includes the cost of whichever baseline point next exceeds the threshold. It should not be

presented as a directly measured same-recall advantage of that magnitude. The companion matched table preserves both estimators and marks targets without coverage, enabling readers to distinguish an operating-region result from a sampling artefact.

### 7.2.3 Interpretation and comparison limits

JHQ combines inexpensive primary screening with selective residual access. This provides a plausible explanation for competitive throughput when a small fraction of scanned candidates needs refinement. The kernel and budget ablations below test parts of that explanation directly. The cross-system curves alone cannot assign the gain to any one component: they also differ in routing, representation, preprocessing, index building and software interfaces. In particular, increasing dimension across these datasets is accompanied by changes in data distribution and dataset size. The stronger OpenAI ratios do not constitute a controlled demonstration that dimensionality causes JHQ's advantage.

Residual comparison limits remain after aligning the main timing boundary. JHQ is called from C++, whereas CAGRA and PQ use Python bindings; small host overhead differences remain even with reused resources. JHQ's output copy targets pageable host memory, whereas baseline outputs use pinned memory. Compilation targets differ, and unrelated host load is not fully controlled. These effects are relevant to small margins. Five rounds characterize repeated execution of the built indexes; they do not estimate variation across independent index builds or independent machines.

## 7.3 Cartesian LUT, Packed Loads and Lane Refinement

### 7.3.1 Exact table factorization and its throughput consequence

Cartesian factorization changes the evaluation of the represented primary distance, rather than training a new quantizer. Splitting an eight-bit Cartesian code into two four-bit components replaces 256 table entries with $2 \times 16 = 32$. With FP32 entries, this changes 1,024 bytes to 128 bytes per corresponding table, an eightfold reduction. The mathematical score is preserved; floating-point evaluation order can still affect

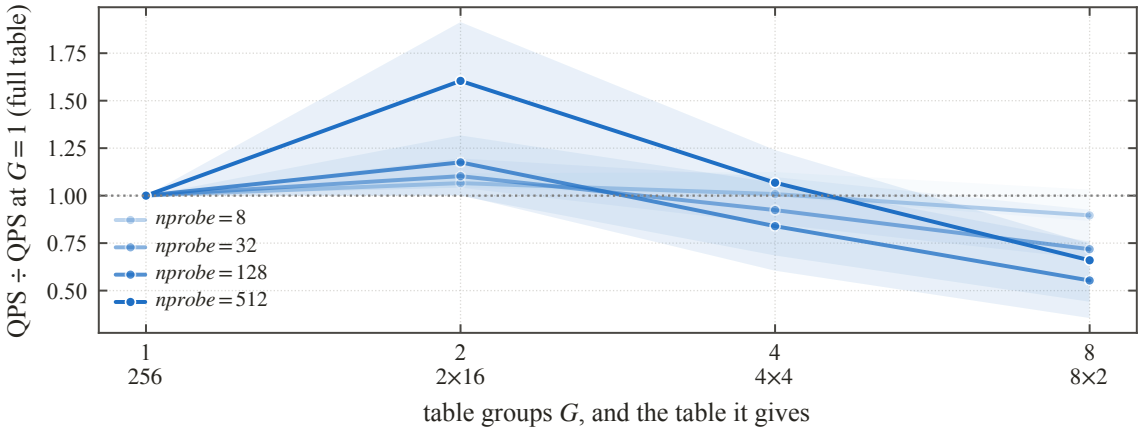



**Figure 7.2:** Completed v71 Cartesian table-grouping experiment, retained from the existing figure set. Group counts describe $g_{\mathrm{LUT}}$, not parallel scan groups. Curves use median QPS normalized to the one-group arm.

the final bits. Smaller tables alone therefore do not justify calling the complete ANN output bit-identical.

Figure 7.2 reuses the completed table-grouping figure, checked against the v71 raw records. It covers six datasets, probe counts 8/32/128/512, fixed $\alpha = 100$, and $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$. Each arm has five timing rounds; batch size is 1,000 except Stella's 999 queries. Two groups have the highest median throughput in 22 of 24 cells. The exceptions are arXiv at 128 probes, where one group is about 0.23% faster, and OpenAI-3072 at eight probes, where four groups are about 0.84% faster. These narrow differences are best treated as near ties. The newer count is 22/24, rather than the historical ADC paper's 23/24.

The two-group/one-group ratio ranges from 0.998 to 1.914. Four and eight groups each require only 16 total entries, but differ in the number and granularity of lookups. Their performance provides a useful control: reducing table storage beyond the two-group design does not necessarily improve search. Additional indexing and accumulation work can outweigh storage savings. The measurements establish the net trade-off; without a separate occupancy or instruction-counter experiment, they do not identify one hardware effect as the sole cause. The maximum reported recall spread within any four-arm cell is $10^{-4}$.

### 7.3.2 Packed access and cooperative residual scoring

The historical packing experiment compares byte-oriented and word-oriented access while holding the dataset, probe count and table grouping fixed. Across six datasets, four probe counts and $g_{\mathrm{LUT}} \in \{1, 2\}$, the 48 pairs yield throughput ratios from 1.012

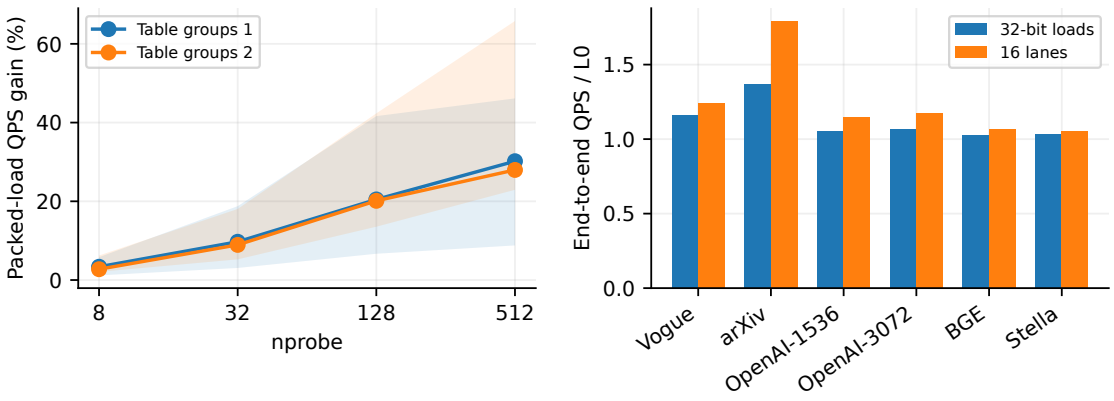



**Figure 7.3:** Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. These panels come from different experiment families.

**Table 7.3:** Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged.

| Dataset | $\alpha$ | $L1/L0$ QPS | $L16/L0$ QPS | Kernel gain |
|---|---|---|---|---|
| arXiv | 100 | 1.368 | 1.789 | 5.97 |
| BGE | 32 | 1.026 | 1.062 | 1.97 |
| OpenAI-1536 | 8 | 1.055 | 1.145 | 3.34 |
| OpenAI-3072 | 4 | 1.063 | 1.171 | 8.81 |
| Stella | 16 | 1.030 | 1.053 | 2.66 |
| Vogue | 64 | 1.161 | 1.240 | 3.72 |

to 1.658, with maximum recall difference $10^{-4}$. Gains are generally smaller at eight probes and larger at 512. Both alternatives already use a subspace-major layout; this experiment supports packed access, not a claim that a physical transpose alone produced the improvement. Each arm has one reported aggregate rate, so this older evidence has weaker uncertainty information than the five-round v71 sweep.

Figure 7.3 and table 7.3 then separate wider loads from lane cooperation in residual refinement. L0 is the original scalar path; L1 changes the load path without distributing a candidate across multiple lanes; L16 assigns 16 lanes to a candidate. The same process rotates arm order across five timing rounds, with three search calls per arm per round. For the twelve ordinary configurations, L16 improves end-to-end throughput by 1.015–1.789×. An additional arXiv configuration at $\alpha = 200$, explicitly a high-budget control, reaches 2.383×; it is not the gain range of the ordinary selected budgets.

The load-only control is bit-identical to L0 in the tested cases, yet already improves arXiv throughput by 1.368×. Lane cooperation adds a larger reduction in refinement time. On arXiv at $\alpha = 100$, the profiled refinement kernel falls from 4,277.5 to 716.8 $\mu$s, a 5.97× gain, while total throughput improves 1.789×. OpenAI-3072's refinement kernel improves 8.81×, but total throughput improves 1.171×. This is consistent with Amdahl's law: acceleration of a small stage cannot remove the cost of routing, screening, final selection or transfers. It also motivates the subsequent scan and probe experiments, which target the remaining work.

Cooperative reductions alter floating-point addition order. The tested multi-lane variants have very small distance differences, with maximum reported relative error about $1.6 \times 10^{-6}$, and occasional changes near ranking boundaries. The evidence therefore supports numerically close scoring with measured end-to-end gains, rather than universal bitwise equivalence. Sanitizer checks cover the recorded refinement and top-$k$ kernels on small query subsets; they are useful implementation checks, not a proof for arbitrary inputs.

## 7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation

### 7.4.1 Global primary selection before performance

The parallel scan experiment addresses a correctness requirement that cannot be evaluated through recall alone. Dividing a query's candidates among blocks must preserve the global primary top-$c$; assigning each block only $c/g_{\text{scan}}$ survivors can discard a globally competitive candidate from a disproportionately productive group. JHQ's v62 path instead retains up to $c$ candidates in every group and merges the local lists. Selection uses the complete key $(D_{\text{P}}, \text{position})$, where position breaks primary-distance ties deterministically.

The local retention rule has a direct justification. A candidate in the global top-$c$ has fewer than $c$ preceding keys in the entire candidate set, and hence fewer than $c$ in its own group. It must therefore appear in that group's local top-$c$. Taking the top-$c$ of the union, with the same total order, recovers the global top-$c$. This statement concerns primary scores over the visited candidates. It does not make IVF routing exact, turn primary scores into raw distances, or make the final ANN answer an exact nearest-neighbour result.

The v62 checker covers four workloads, batches 8/32/256/1,000 and group counts 1/2/4/8, giving 64 comparisons. It reports no non-tie primary selection differences from the original one-block path, zero change in compared primary distances, and bit-identical final top-10 identifiers and distances for every used query. The original path sorts on distance alone, so its primary cutoff can select a different member of a tie; these differences do not violate the new complete-key specification. They also did not change final outputs in this particular experiment.

The number of used queries deserves attention. The harness submits only complete batches: out of a 1,000-query pool it uses 992 queries for batch 32 and 768 for batch 256, whereas batch 8 and batch 1,000 use all 1,000. Correctness counts and timing populations therefore depend on the batch regime. They cannot be described as the identical 1,000-query test in every row. Recorded memcheck, racecheck and synccheck runs provide additional checks on smaller subsets and on the named scan/merge kernels.

### 7.4.2 Shared bounds preserve the selection target

Keeping a full local top-$c$ is safe but potentially expensive. In v63, groups of one query publish a shared bound using the same complete key. Once any group has established $c$ keys no greater than a bound, a candidate worse than that bound cannot enter the global top-$c$. A stale, looser bound can cause extra admissions, but cannot legitimately exclude a better candidate. This requires that a published bound be backed by enough valid candidates and that atomic updates preserve the complete-key ordering; an approximate distance threshold alone would not establish the same tie behaviour.

The implementation still computes every scanned candidate's complete primary score before deciding admission. The bound reduces selection, compaction, sorting and downstream merging work; it does not prune primary distance evaluation or avoid reading every rejected primary code. This scope matters when interpreting an apparent scan-stage speedup. A faster scan stage need not mean fewer evaluated distances.

The dedicated v63 checks compare shared-bound groups 2/4/8 with their independent-bound counterparts across the four workloads and four batch sizes. All 48 comparisons report identical primary top-$c$ and final outputs. Five-round paired throughput measurements range from about 0.994 to 1.562× relative to independent bounds. Thus sharing substantially helps some expensive cases but is not uniformly

**Table 7.4:** v69/v65 end-to-end throughput ratios for the exact FP32 probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Values above one favour v69.

| Dataset | Batch 8 | 32 | 256 | 1,000 |
|---|---|---|---|---|
| arXiv | 1.074 | 1.063 | 1.035 | 1.034 |
| Vogue | 1.109 | 1.103 | 1.052 | 1.041 |
| OpenAI-3072 | 1.167 | 1.117 | 1.055 | 1.050 |

faster. On arXiv at $\alpha = 200$, batch 1,000 and eight groups, the profiled merge cost falls from approximately 2,629 to 471 $\mu$s. The result supports reducing excess intermediate work rather than claiming that more scan groups are always beneficial.

A rejected alternative was to allocate smaller static local quotas. The campaign's distribution measurements place 46–81% of the global survivors in the first candidate group for the pilot workloads. Equal quotas would frequently require a correctness fallback, and calibrated unequal quotas still leave the dominant group on the critical path. These observations motivated the exact shared bound. They are pilot measurements of those workloads, not a theorem that every possible quota policy is inferior.

### 7.4.3 Exact probe selection and the limits of pipeline identity

Once screening becomes faster, IVF probe selection becomes a larger fraction of small-batch time. The v69 probe primitive changes how the smallest centroid scores are selected while retaining FP32 scoring. Its synthetic checker covers 1,792 cases and reports zero failures. Checks of the real arXiv, Vogue and OpenAI-3072 workloads find identical probe identifiers and identical primary top-$c$ positions and distances for all 1,000 queries. These are more specific observations than a single unchanged recall number: they test the routing and screening outputs directly.

Table 7.4 shows end-to-end improvements of 1.034–1.167×. At batch 8, the profiled probe stage falls from 56.5 to 22.6 $\mu$s on arXiv, 52.7 to 14.7 $\mu$s on Vogue, and 77.8 to 16.7 $\mu$s on OpenAI-3072. Smaller full-batch gains are expected because the unmodified stages account for more of total elapsed time. These are within-family ratios; they should not be multiplied by the preceding v62 or refinement gains as if all measurements shared a single baseline.

The archived Vogue comparison explicitly reports an identity failure: v65 recall is 0.9642 and v69 recall is 0.9644, although the recorded probe and primary selections match. Within v69, the scheduled and one-block paths return bit-identical final

outputs for all 1,000 queries. The cross-binary observation therefore supports exact routing equivalence in the tested case, but does not support bitwise identity of the complete pipeline across versions. Later timing runs show the same Vogue recall for both versions, which is consistent with run-dependent effects but does not establish the cause of the earlier difference. The anomaly is retained rather than attributed to an unverified mechanism.

Finally, the optional v70 FP16 experiment is a separate approximation. Its probe sets match FP32 exactly on 936/1,000 arXiv queries, 979/1,000 Vogue queries and 998/1,000 OpenAI-3072 queries, with very high mean set overlap. High overlap and small recall changes do not make the primitive exact. Those results are excluded from the main E1 frontier and from the exact-probe claim.

## 7.5 Held-out Budget Quality, Throughput and Payback

### 7.5.1 Unlabelled selection and a labelled diagnostic oracle

The budget experiment fixes one trained index per dataset–probe cell and compares fixed budgets, sampled calibration, full-pool calibration, same-sample enumeration and an offline oracle. The six datasets are tested at 128 and 512 probes. Even-indexed queries form a calibration pool of 500; odd-indexed queries form a disjoint held-out pool of 500, except Stella, which has 499. Samples of $S \in \{32, 64, 128\}$ are drawn from the calibration pool without replacement within each draw, using a fixed random seed, with 64 draws per size. Repeated draws reuse a fixed finite query pool; they are not 64 independent samples of future workload drift.

The descending grid is $A = (200, 100, 64, 32, 16, 8, 4, 2)$. Calibration compares candidate top-10 identifier sets with the $\alpha_{\max} = 200$ reference. For sample $\mathcal{Q}_S$, the acceptance statistic is

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(10 - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{200}(q)|\right), \qquad D_S(\alpha) \leq 1. \tag{7.1}$$

The allowance is one disagreeing result slot over the entire sample, not one per query and not a ground-truth recall tolerance. No labels are supplied to the deployed rule. The reference is itself approximate, so perfect agreement can reproduce its errors.

**Table 7.5:** Held-out quality at $S = 128$. Rule $\alpha$ is the median across 64 draws; oracle $\alpha$ uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; $L_{95}$ is the logged lower-order 95th-percentile loss estimate. $R_{100}$ is fixed-budget recall.

| Dataset | Probe | $\alpha$: rule/oracle | $R_{100}$ | $\overline{R}_{\text{rule}}$ | Losses | $L_{95}$ |
|---|---|---|---|---|---|---|
| Vogue | 128 | 64 / 64 | 0.9680 | 0.9677 | 5/64 | 0.0022 |
| arXiv | 128 | 100 / 200 | 0.9748 | 0.9751 | 40/64 | 0.0024 |
| OpenAI-1536 | 128 | 8 / 8 | 0.9280 | 0.9279 | 4/64 | 0.0000 |
| OpenAI-3072 | 128 | 4 / 4 | 0.9252 | 0.9252 | 0/64 | 0.0000 |
| BGE | 128 | 32 / 32 | 0.9198 | 0.9190 | 1/64 | 0.0008 |
| Stella | 128 | 16 / 16 | 0.9892 | 0.9884 | 6/64 | 0.0030 |
| Vogue | 512 | 64 / 64 | 0.9916 | 0.9915 | 2/64 | 0.0002 |
| arXiv | 512 | 200 / 200 | 0.9878 | 0.9885 | 32/64 | 0.0016 |
| OpenAI-1536 | 512 | 8 / 8 | 0.9740 | 0.9738 | 4/64 | 0.0000 |
| OpenAI-3072 | 512 | 4 / 8 | 0.9704 | 0.9702 | 0/64 | 0.0002 |
| BGE | 512 | 64 / 64 | 0.9670 | 0.9661 | 24/64 | 0.0014 |
| Stella | 512 | 16 / 16 | 0.9954 | 0.9945 | 7/64 | 0.0030 |

The offline oracle has a different role. After observing held-out recall for all eight fixed budgets, it selects the fastest measured arm within $10^{-3}$ of the best measured held-out recall. This is a diagnostic benchmark, not an implementable label-free decision. Likewise, "full pool" means applying the same calibration search to all 500 calibration queries; it does not mean enumerating every budget with held-out labels. The distinctions prevent an apparent oracle match from being mistaken for a quality guarantee.

### 7.5.2 Budget quality and sampling risk

Table 7.5 reports the $S = 128$ result. The median selected budget matches the oracle budget in ten of twelve cells; it differs on arXiv at 128 probes and OpenAI-3072 at 512. The selected budgets span 4–200, indicating substantial workload dependence. However, a median budget and a mean recall describe different aspects of the draw population. They need not correspond to the performance of one arm. In addition, mean recall in the table is reconstructed from the printed best recall minus printed mean loss, so its four-decimal precision is approximate.

The important counterexample is arXiv. At 128 probes, 40/64 draws lose more than $10^{-3}$ recall even though mean loss is only 0.0009; at 512 probes, 32/64 draws cross the threshold. BGE at 512 probes has 24/64 threshold crossings and a 95th-percentile loss of 0.0014. Stella has fewer crossings but a 95th-percentile loss of 0.0030 at both probe depths. Thus mean quality, violation frequency and tail magnitude

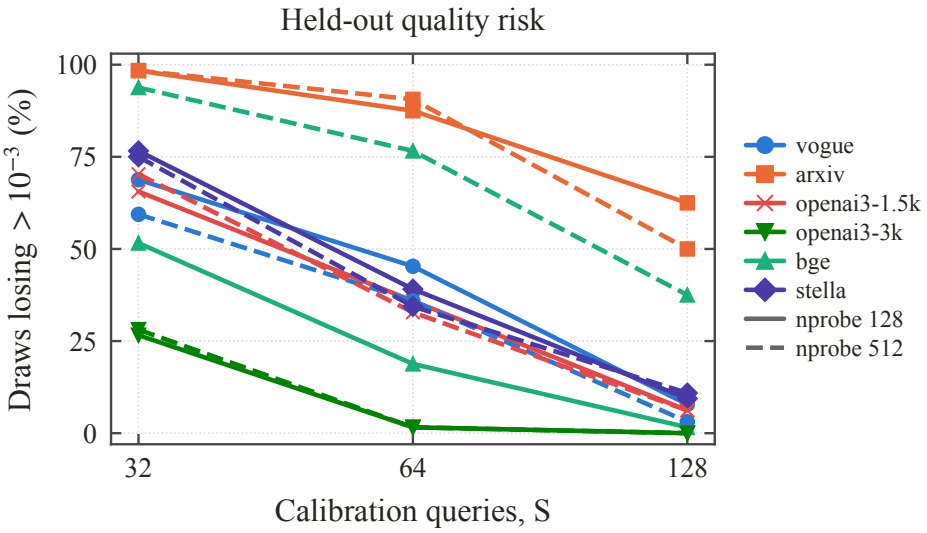



**Figure 7.4:** Existing v69 held-out risk figure, checked against the budget log. The vertical axis counts calibration draws whose held-out recall loss exceeds $10^{-3}$. Increasing $S$ also tightens the allowed slot disagreement fraction because the threshold in equation (7.1) remains one.

must be read together. OpenAI-3072 has no observed threshold crossings in these draws, which supports a favourable result on this pool, not a zero-risk statement about unseen queries.

The percentile convention also matters for this small sample of draws. The harness reports the sorted loss at zero-based index $\lfloor 0.95(64-1) \rfloor = 59$, rather than interpolating between order statistics. Consequently, four nonzero tail losses can coexist with a printed $L_{95} = 0$, as on OpenAI-1536. The table preserves this logged statistic and the exact crossing count; zero in that column does not mean every draw has zero loss.

Figure 7.4 shows the effect of sample size. A larger sample usually makes selection more conservative, but the comparison changes two things at once: sample coverage increases and the normalized allowance $1/(10S)$ decreases. The figure cannot isolate those effects. Same-sample enumeration agrees with bisection in all 2,304 draws ($12 \times 3 \times 64$), while taking approximately 1.75–2.01 times as long in the printed logs. Bisection therefore realizes the same sampled criterion more cheaply on this grid. It does not fix the criterion's held-out failures. Nor does empirical agreement establish universal monotonicity: the deployed final selector retains a bounded number of thread-local candidates, so the exact-selection assumptions needed for a general monotonicity argument are not assured.

A fixed budget is also imperfect. Table 7.6 evaluates all eight fixed choices against the diagnostic quality target. Fixed $\alpha = 100$ qualifies in ten of twelve cells; $\alpha = 200$

**Table 7.6:** Universal fixed-budget controls over the twelve held-out cells. Qualifying means within $10^{-3}$ of the best tested recall, using printed precision. QPS penalty is the largest fractional deficit relative to the cell's quality-constrained oracle, irrespective of whether that fixed arm itself qualifies.

| $\alpha$ | Qualifying cells | Worst recall loss | Largest QPS penalty |
|---|---|---|---|
| 2 | 0/12 | 0.3100 | 11.2% |
| 4 | 2/12 | 0.1774 | 2.3% |
| 8 | 4/12 | 0.0930 | 2.1% |
| 16 | 6/12 | 0.0408 | 4.4% |
| 32 | 7/12 | 0.0170 | 9.1% |
| 64 | 10/12 | 0.0050 | 26.6% |
| 100 | 10/12 | 0.0016 | 40.9% |
| 200 | 12/12 | 0.0000 | 57.1% |

qualifies in all twelve but can sacrifice substantial throughput. Full-pool calibration is not identical to the oracle either: on Vogue it selects 100 at both probe depths while the oracle selects 64. Agreement-based selection and fastest quality-constrained selection solve different problems, even with a larger calibration sample.

### 7.5.3 Estimated throughput and calibration economics

Unlike E1, this family times each fixed arm with a warm-up and three search calls. The rule's reported QPS is then the arithmetic mean of the already measured QPS values of its 64 selected arms. It is not a new timing of a service repeatedly calibrating and executing the policy. At $S = 128$, this selected-arm estimate improves over fixed $\alpha = 100$ in ten cells, by 1.028–1.691×. On arXiv the ratios are 0.908 and 0.917: mean recall improves, but throughput falls. Those cells have no throughput payback.

Calibration costs 3.9–21.3 ms in these logs. This includes reference search and agreement checks, with any included search setup inside the measured calls, but excludes gathering sample vectors into the input buffer. For an equal-sized subsequent batch, define

$$B^* = \frac{T_{\text{cal}}}{T_{100} - T_{\text{rule}}}, \qquad T_{\text{rule}} < T_{100}. \tag{7.2}$$

Using $T_{\text{rule}} \approx n/\overline{Q}_{\text{rule}}$ gives the estimates in table 7.7, approximately 2.4–55.4 held-out-sized batches for the favourable cells. The existing payback figure uses 500-query equivalents even for Stella; the table corrects its population to 499. This changes Stella's estimate by approximately 0.2%, rather than introducing a new experiment.

The calculation is optimistic. In general, $1/\mathbb{E}[Q] \leq \mathbb{E}[1/Q]$, so inverting mean QPS understates mean execution time when calibration draws choose arms with

**Table 7.7:** v69 calibration economics, $S = 128$. Rates are kQPS; rule rate is mean selected-arm QPS. Payback uses the actual held-out pool size (500, or 499 for Stella). "Never" means estimated selected execution is slower than fixed $\alpha = 100$, so throughput cannot repay calibration.

| Dataset | Probe | $Q_{100}$ | $\overline{Q}_{\text{rule}}$ | Ratio | Cal. ms | Batches |
|---|---|---|---|---|---|---|
| Vogue | 128 | 202.9 | 224.4 | 1.106 | 4.0 | 17.0 |
| arXiv | 128 | 207.9 | 188.8 | 0.908 | 4.2 | Never |
| OpenAI-1536 | 128 | 167.5 | 268.5 | 1.603 | 3.9 | 3.5 |
| OpenAI-3072 | 128 | 85.7 | 145.0 | 1.691 | 5.7 | 2.4 |
| BGE | 128 | 116.2 | 131.2 | 1.130 | 6.5 | 13.2 |
| Stella | 128 | 95.7 | 110.3 | 1.153 | 7.5 | 10.8 |
| Vogue | 512 | 100.1 | 102.9 | 1.028 | 7.5 | 55.4 |
| arXiv | 512 | 83.1 | 76.2 | 0.917 | 9.7 | Never |
| OpenAI-1536 | 512 | 86.6 | 115.0 | 1.327 | 8.8 | 6.2 |
| OpenAI-3072 | 512 | 42.8 | 52.1 | 1.219 | 13.1 | 6.2 |
| BGE | 512 | 49.2 | 51.1 | 1.039 | 15.8 | 41.2 |
| Stella | 512 | 34.6 | 36.0 | 1.041 | 21.3 | 37.1 |

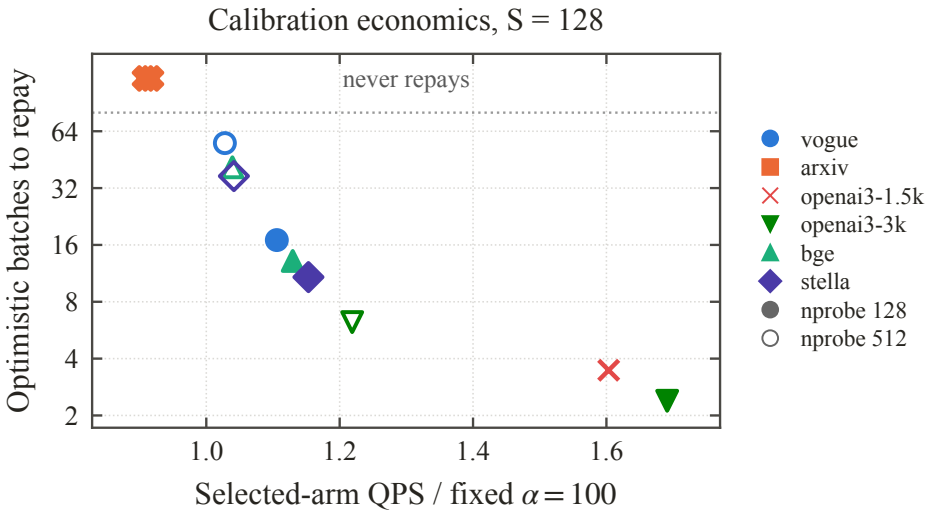



**Figure 7.5:** Existing v69 payback figure. Payback is an optimistic estimate using mean selected-arm QPS and 500-query equivalents. ArXiv has no throughput payback. Table 7.7 retains actual held-out counts.

different rates. Sample gathering, future query drift and recalibration overhead would further affect amortization. A policy reused for a long stationary workload can benefit; a short job or a changing query population may not. The present experiment measures neither drift detection nor a production recalibration schedule.

## 7.6 Batch Regimes, G Scheduling and Workload Skew

### 7.6.1 Parallelism versus repeated selection work

The scan experiments reveal two execution regimes. With one block per query, batch 8 exposes only eight blocks on a 170-SM device. Candidate partitioning increases the available block parallelism. In the v62 experiment, eight groups improve end-to-end throughput by 2.06–2.76× at batch 8. Profiling supports a stage-level explanation: arXiv's scan falls from about 849 to 216 $\mu$s, Vogue's from 1,059 to 246 $\mu$s, and OpenAI-3072's from 728 to 155 $\mu$s. The additional merges cost approximately 16.7, 9.1 and 3.7 $\mu$s, respectively. The net reduction is large even after paying for merging.

At batch 1,000 there are already enough independent queries to expose many blocks. Additional groups repeat table staging, local-buffer admission and sorting, and produce extra merge traffic. The unshared v62 path can fall to about 0.46× its one-block baseline on arXiv at $\alpha = 200$. This opposing result is central to the design: parallel decomposition solves insufficient concurrency, but incurs work that can dominate once concurrency is adequate. Shared bounds reduce that extra work but do not eliminate the regime change.

### 7.6.2 A fixed rule evaluated on further configurations

The integrated v65 scheduler chooses a scan-group count using batch size, block size and a workload-skew estimate. Its fill rule relates the potential resident threads to a fixed 0.6 fraction of the device's thread capacity. Its skew rule requires at least four groups when the coefficient of variation of candidates per query is at least 0.3. The larger suggested group count is used, with the one-group case mapped to the original one-block scan. The skew statistic is estimated once from 512 database vectors used as pseudoqueries, not from ground-truth labels or the evaluated query pool. The constants were chosen on four workload configurations before the additional configurations were measured.

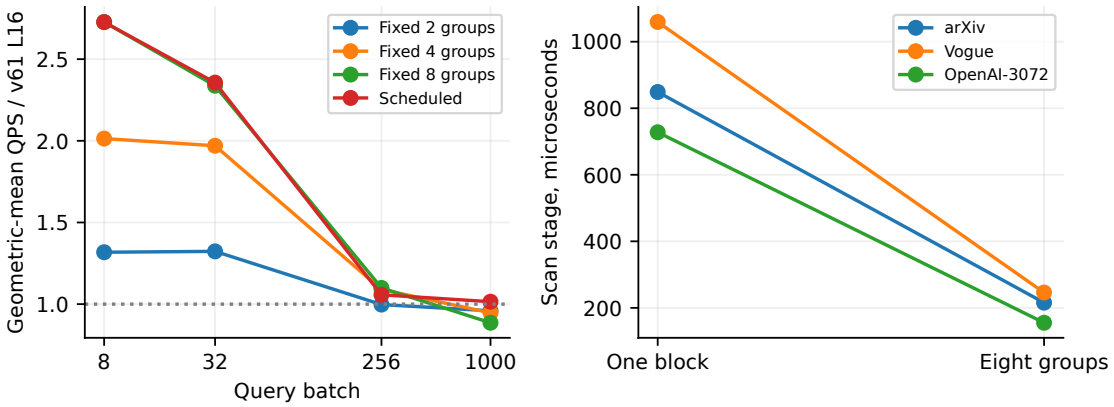



**Figure 7.6:** Batch-dependent execution. Left: integrated shared-bound fixed group counts and scheduled execution relative to v61 L16; the full-batch population has 11 cells, versus 13 at smaller batches. Right: v62 batch-8 profiled scan time; merge time is additional. The panels separate application throughput from the stage evidence explaining small-batch gains.

**Table 7.8:** Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. "Held out" excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment's held-out query split.

| Batch | Cells | $g_s = 2$ | $g_s = 4$ | $g_s = 8$ | Scheduled | Held out |
|---|---|---|---|---|---|---|
| 8 | 13 | 1.318 | 2.014 | 2.728 | 2.727 | 2.930 |
| 32 | 13 | 1.323 | 1.970 | 2.337 | 2.356 | 2.560 |
| 256 | 13 | 0.997 | 1.090 | 1.099 | 1.057 | 1.041 |
| 1000 | 11 | 0.959 | 0.947 | 0.886 | 1.015 | 1.012 |

Table 7.8 recomputes the results from the raw timing arrays. The scheduled geometric-mean gains are 2.727×, 2.356×, 1.057× and 1.015× at batches 8, 32, 256 and 1,000. There are thirteen configurations at the first three batch sizes, including an additional high-budget arXiv case, but only eleven at batch 1,000: Stella's 999-query pool cannot provide a complete 1,000-query batch. The held-out subsets contain nine and seven configurations, respectively. A plot connecting these means summarizes regimes rather than a perfectly balanced repeated-measures population.

Fixed eight-group execution matches the scheduled mean closely at batch 8 but falls to 0.886× at batch 1,000. Scheduling largely retains the small-batch gain while avoiding that full-batch loss. It is nevertheless a heuristic, not an oracle. At batch 256 its worst ratio of best fixed throughput to scheduled throughput is 1.217, on a BGE configuration. Candidate-count CV does not capture every cost of list traversal, selection or merging. This failure identifies a limitation of the chosen feature rather than evidence that execution scheduling is unnecessary.

Vogue illustrates why batch size alone is insufficient. Its measured candidate-count CV is about 0.51, compared with roughly 0.08–0.11 for the original other workloads, and its heaviest query has around 2.2 times the mean candidate count. Splitting candidates can shorten a slow query that would otherwise determine completion of a synchronous batch. This explains why Vogue can still benefit from multiple groups at larger batches, although the CV observations are workload evidence rather than a general causal isolation of skew.

Finally, correctness is checked independently of timing. Across 250 variant/configuration comparisons and 234,320 query comparisons, the integrated campaign reports no non-tie primary-selection differences and no different final identifier sets. Final order can differ at ties; the entire campaign should not be described as universally bit-identical. The throughput results concern sequential, synchronous submissions and do not establish tail latency under an open-loop arrival process.

**Table 7.9:** Cold JHQ construction and separately measured resident memory. Train/add come from one recorded cold build per dataset. Memory comes from v71's two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. "Residual" is the modelled residual-code share of the measured used total, not the fraction read per query.

| Dataset | Train s | Add s | Total s | Used GiB | Raw GiB | Residual |
|---|---|---|---|---|---|---|
| Vogue | 1.06 | 0.27 | 1.33 | 1.47 | 2.67 | 45.4% |
| arXiv | 0.59 | 0.52 | 1.10 | 2.58 | 6.45 | 62.5% |
| OpenAI-1536 | 0.64 | 0.38 | 1.01 | 2.39 | 5.72 | 59.7% |
| OpenAI-3072 | 1.34 | 0.65 | 1.99 | 4.14 | 11.43 | 69.0% |
| BGE | 3.75 | 3.67 | 7.42 | 11.93 | 38.50 | 80.7% |
| Stella | 3.74 | 6.67 | 10.40 | 20.44 | 67.81 | 82.9% |

## 7.7 Cold Construction, Resident Memory and Tested Scale

### 7.7.1 Cold builds and the lifecycle boundary

The principal search sweeps reuse trained-state caches. Loading that state and adding vectors is not a cold build. Separate logs explicitly train with an empty trained-state cache and then encode/add the database; table 7.9 reports their recorded training and add times. The resulting totals range from about 1.01 s for OpenAI-1536 to 10.40 s for Stella. Stella's 3.74 s training and 6.67 s add phases also illustrate that neither phase can be inferred reliably from the other or from dataset size alone.

Cold here refers to learned index state. The input loader can reuse memory-mapped FP32 data, and the reported train-plus-add interval does not include reading the original exported files from storage. The scheduler's pseudoquery calibration is performed after add and is outside that total. A deployment lifecycle would also account for data preparation, transfer where excluded by a given API, calibration, serialization and subsequent query time. The recorded totals are useful construction measurements within their boundary, not a complete end-to-end ingestion service cost.

The comparison figure reports alternative construction configurations alongside these cold builds. Its method-specific ranges are not confidence intervals over equivalent repeated builds. Training samples, index parameters and representations differ, and JHQ has only one cold construction record per dataset. For RaBitQ, the recorded build function is separated from serialization/deserialization, although the benchmark requires that round trip before search. Omitting that extra step is

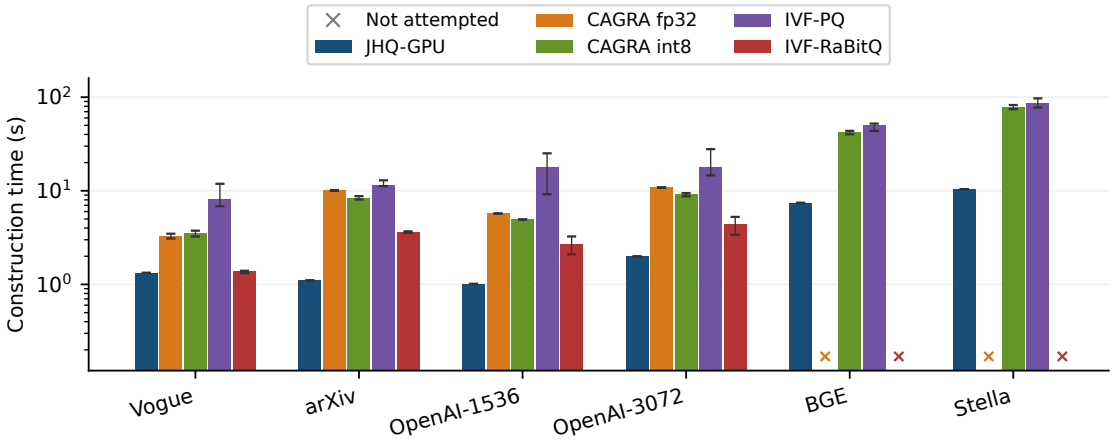



**Figure 7.7:** Recorded construction costs, with CAGRA int8 included. JHQ is one cold train-plus-add record; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic.

appropriate only when clearly labelling the bar as build-function time; it does not measure time until the first usable search.

On Stella, successful isolated CAGRA int8 builds take approximately 74.55 and 82.45 s including dataset quantization, depending on graph degree; the measured IVF-PQ build median is 86.59 s across its three configurations. JHQ's 10.40 s record is substantially shorter under the reported boundaries, but does not establish a universally fastest builder or equal downstream recall. Construction speed, searchable quality and resident cost are separate deployment dimensions.

### 7.7.2 Resident allocation and where compression helps

Figure 7.8 retains the completed memory-component figure. Its markers are device memory in use at the recorded resident measurement, not index-only allocations. The stacked components model primary and residual codes, correction factors and identifiers, centroids, per-batch tables, candidate buffers and list-group scratch. The remaining measured allocation stays explicitly unattributed; it may include runtime context, allocator effects and allocations omitted by the model. Assigning that remainder to residual codes would overstate the representation cost.

Residual codes dominate the large datasets: approximately 80.7% of BGE's 11.93 GiB and 82.9% of Stella's 20.44 GiB used totals. The corresponding raw FP32 vector arrays alone would require 38.50 and 67.81 GiB. JHQ therefore enables the tested resident search on one card by retaining a compressed hierarchy. Lowering $\alpha$

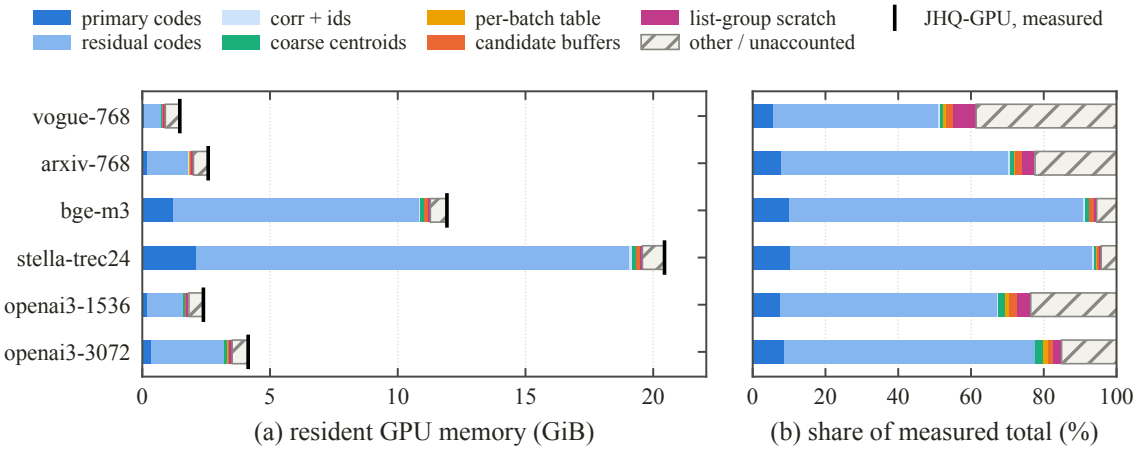



**Figure 7.8:** Completed resident-memory decomposition, verified against the v71 two-group measurements. Component sizes are modelled, markers are measured used totals, and hatching retains the unattributed remainder. This is neither a construction-peak trace nor an index-only measurement.

reduces residual codes accessed and scored per query, but leaves the stored residual array allocated. Similarly, Cartesian LUT factorization reduces query-specific table storage, not the database's residual-code precision.

Workspace also matters. The reported list-group scratch is approximately 93.8 MiB in these $\alpha = 100$ measurements, reserved for a batch capacity of 1,024 and up to eight groups even when the selected execution uses the one-block path. A smaller actual group count therefore does not imply that the allocation shrank proportionally. Conversely, these resident samples do not capture temporary construction peaks. The baseline harness also initializes some search memory pools after its index-memory measurement, so an after-build index delta cannot be used as a complete peak process-memory budget across methods.

### 7.7.3 Large-dataset feasibility and competitive regions

BGE contains 10,091,524 vectors and Stella 17,776,615, both with 1,024 dimensions and 32,768 IVF lists. Stella's queries are sampled from the corpus; its result therefore characterizes that benchmark rather than an independently collected external-query distribution. Figure 7.9 includes successful CAGRA int8 and IVF-PQ measurements. Full-precision CAGRA, RaBitQ configurations and FP32 brute force were not attempted in the large-scale comparison when their computed storage requirements exceeded the available device capacity. Those omissions are planned feasibility exclusions, not observed out-of-memory errors or measured recall failures.

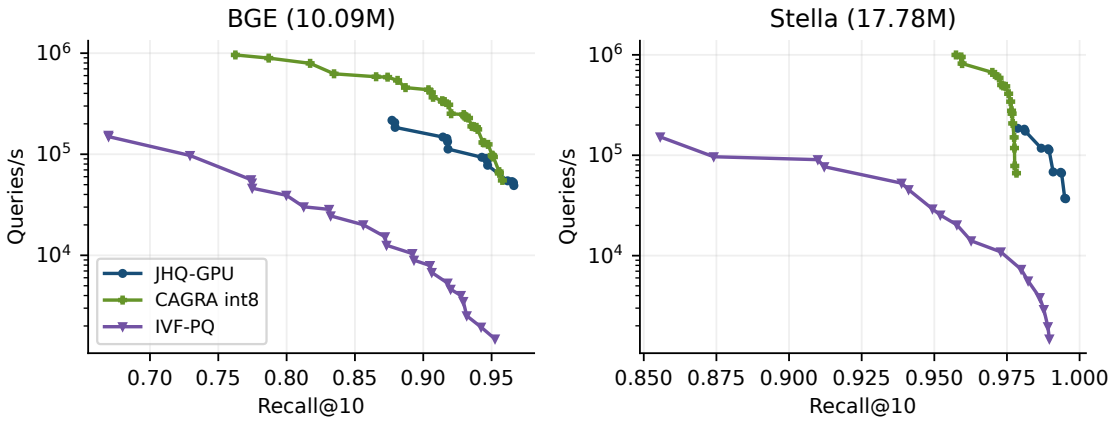



**Figure 7.9:** Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE uses 1,000 queries and Stella 999. An endpoint is the end of the tested grid, not an intrinsic recall limit.

An earlier large-dataset baseline attempt retained a previous index through a closure, contaminating the apparent memory feasibility of the next build. Successful isolated reruns supersede that attempt. In particular, CAGRA int8 does fit Stella in the tested configurations; claiming that all graph baselines exceed the card capacity would be incorrect. The corrected frontier shows a different trade-off: CAGRA int8 is highly competitive at lower recall but reaches approximately 0.9579 on BGE and 0.9783 on Stella in its measured sweep. JHQ reaches approximately 0.9663 and 0.9952, respectively. IVF-PQ reaches about 0.9525 and 0.9896. These maxima describe coverage of the recorded grids.

The result is evidence of tested scale and useful high-recall operating regions, rather than arbitrary-size scalability. The largest completed search uses 17.78 million vectors on one GPU; there is no multi-GPU experiment, no scaling curve holding data distribution fixed and no construction-peak proof near the card's capacity. Likewise, Stella's 999-query search results come from its explicit rerun; an empty 1,000-query search list in a cold-build log cannot supply a throughput point.

## 7.8 Numerical Robustness, Anomalies and Recall Attribution

### 7.8.1 Quality resolution and repeated execution

With $k = 10$ and 1,000 queries, one recovered neighbour changes aggregate Recall@10 by $10^{-4}$; with 500 held-out queries the resolution is $2 \times 10^{-4}$. Stella's 499-query split

has a slightly coarser resolution. This discretization is not an uncertainty interval, but it helps interpret the small differences reported above. A fourth-decimal change may represent only one or two result slots, while a small mean loss can still hide several queries with substantially changed answers.

Repeated timing calls must also be separated from repeated index builds. The E1 parser records 369 baseline operating points with a false identifier-determinism flag: 170 RaBitQ, 80 IVF-PQ, 68 CAGRA int8, 50 CAGRA fp32 and one brute-force point. This flag can include result order changes at ties; it does not by itself imply changed identifier sets or invalid output. No such point is discarded solely for that flag. The raw logs retain first/last recall and set/order diagnostics. JHQ does not populate this same baseline flag, so its absence cannot be counted as a uniform cross-method determinism test.

The five-round throughput summaries retain variability within a fixed build. Minima and maxima are descriptive ranges, not confidence intervals, and they do not account for training variation. Small throughput margins such as the two LUT near ties, or the arXiv–RaBitQ comparison at recall 0.97, consequently receive limited interpretive weight. Larger repeated gains with a matching stage-time reduction provide stronger evidence for an execution improvement than a small frontier displacement alone.

### 7.8.2 Exactness is attached to a particular stage

Several distinct claims must remain separate. Cartesian factorization preserves the represented score algebraically. Packed loads preserve the logical code values. Complete-key local retention and merging recover the exact primary top-$c$ of the visited candidates. The shared bound preserves that selection target. The FP32 probe primitive passes its synthetic and real-input equivalence checks. None of these establishes exact nearest-neighbour answers in the original vector space.

Lane reductions can alter rounding; primary cutoff ties can change which survivor is refined; approximate final selection can change a boundary result. The archived Vogue cross-version discrepancy shows why a pipeline-wide identity claim requires its own check. Similarly, the FP16 routing experiment demonstrates that an almost unchanged mean recall is insufficient evidence of exact probe equivalence. The main correctness conclusions are therefore stated at the stage and input coverage actually tested.

Recall loss can enter at routing, primary screening, residual scoring or final selection. Increasing probes expands routing coverage; increasing $\alpha$ gives more primary survivors access to residual scoring; neither restores information lost by every later approximate stage. The budget sweep directly measures the consequence of changing survivor count while holding the index and probe configuration fixed. It does not decompose every missing neighbour into a unique cause. A complete per-query attribution would require ground-truth coverage checks after routing, after primary retention and after refinement, with an exact final-selection reference on the same candidate set. That full attribution is not established by the experiments used here, and no percentages are assigned to unmeasured sources of loss.

### 7.8.3 Scope of the research findings

Taken together, the evidence supports representation-aware execution, exact parallel primary selection and workload-specific refinement budgets as useful components of GPU JHQ. It also identifies their boundaries: more parallel groups can increase work, smaller tables can require more lookups, sampled agreement can generalize poorly, and compressed residency retains a substantial residual-storage cost. These are informative negative results because they constrain the mechanisms offered to explain the positive results.

The claims apply to the exported squared-L2 workloads, top-10 search, the recorded query pools and the tested RTX 5090 environment. They do not establish behaviour for another GPU, a different $k$, changing query distributions, online updates or concurrent service arrivals. Within that scope, the strongest evidence combines a controlled change, explicit correctness checks, a repeated end-to-end gain and a measured reduction in the targeted stage. The following discussion can therefore build on verified operating regions and mechanisms without treating unmeasured deployment properties as established results.

Chapter 8

# Discussion, Limitations and Future Research

**TO WRITE:** Synthesize supported findings, analyse two relevant negative designs, and state internal and external validity limits.

## 8.1 Three Cross-Layer Insights

**TO WRITE:** Develop the supported links between representation and execution, parallelism and repeated work, and execution cost and the value of budget reduction.

## 8.2 Two Deep Negative-Result Case Studies

**TO WRITE:** For two case studies, state hypothesis, alternatives, test conditions, observations, and stopping decision. Label isolated GPU, offline, estimated, and untested evidence.

## 8.3 Threats to Internal and External Validity

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 8.4 Original Testable Research Directions

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 8.5 Conditional Deployment Recommendation

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 9

# Conclusion

**TO WRITE:** Answer the research questions using only results already established. Do not introduce new measurements or contributions.

## 9.1 Answers to the Four Research Questions

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

## 9.2 Closing Principle and Outlook

**TO WRITE:** Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

# Bibliography

[1] Jiaxin Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[3] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[4] Jianyang Shi, Jianyang Gao, Jiaxin Xia, Tamás B. Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

Appendix A

# Reproducibility and Experiment Manifest

## A.1 Source and Build Environment

**TO WRITE:** Record immutable source commits, compiler options, GPU and host details, dependencies, and how the binaries were built. Distinguish the thesis repository from the implementation repository.

## A.2 Data and Trained States

**TO WRITE:** Document dataset identifiers, preprocessing, query and GT hashes, split rules, trained-state cache identities, and exclusions.

## A.3 Result-to-Figure Mapping

**TO WRITE:** Map every empirical figure to its raw files, analysis script, configuration, metric definition, and reproduction command. Explain exactly what cold construction includes.

Appendix B

# Supplementary Results and Correctness Checks

## B.1 Complete Parameter Grids

**TO WRITE:** List method-specific tuning grids, invalid or unavailable settings, and reasons for unattempted configurations.

## B.2 Correctness and Numerical Checks

**TO WRITE:** Include reference comparisons for encoding, LUT factorisation, primary selection, probe selection, and refinement. Separate numerical, algorithmic, and memory-safety evidence.

## B.3 Additional Results and Negative Designs

**TO WRITE:** Include secondary results with evidence status: integrated system, isolated GPU measurement, offline replay, analytical estimate, or untested idea.

Appendix C

# Contributions and Tool-Use Record

**TO WRITE:** Use the form of disclosure required by Canvas. The tables below are a record template, not an attestation that any particular verification was performed. Replace all entries with a truthful account.

## C.1 Contribution Record

| Work component | Contributor and role | Supporting record |
|---|---|---|
| *To complete* | *To complete* | *To complete* |

## C.2 Generative-AI Assistance

| Tool and date | Purpose and scope | Prompt record and human review |
|---|---|---|
| *To complete* | *To complete* | *To complete* |

Appendix D

# Template Examples — Draft Only

This appendix demonstrates the available formatting. It is omitted when `\draftthesisfalse` is selected in `config/metadata.tex`.

## D.1 Citations and Cross-References

Use `\citet{jhq}` for a narrative citation: Han et al. [1]. Use `\citep{pq,cagra}` for a parenthetical citation [2, 3]. Examples: figure D.1, table D.1, and equation (D.1). The bibliography entries are copied from the existing ADC source; review their metadata before submission.

## D.2 A Source-Native Diagram

## D.3 Tables

## D.4 Equations and Proofs

$$D(\alpha) = \sum_{q \in Q_s} \left[ k - \left| R_\alpha(q) \cap R_{\max}(q) \right| \right]. \tag{D.1}$$

The environments `definition`, `lemma`, `proposition`, and `proof` are available. For example:

**Definition D.1** (Strict total ranking key)**.** A ranking key specifies how to order unequal scores and how to break ties.

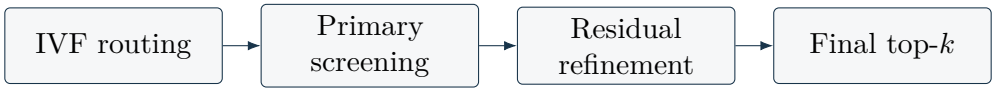



**Figure D.1:** Formatting example: a schematic query path. Replace or extend it with the verified execution boundaries of the implementation.

**Table D.1:** Example manifest fields. This table contains no experimental results.

| Field | What to record |
|---|---|
| Source | Immutable commit and build configuration |
| Index | Trained-state identity and input hashes |
| Timing | Host-to-host or device-resident boundary |
| Quality | Metric, ground truth, and held-out split |
| Evidence | Raw log and analysis-script path |

**Table D.2:** Pseudocode layout example.

| Line | Operation |
|---|---|
| 1 | Read the input and validate the stated preconditions. |
| 2 | Compute the intermediate result. |
| 3 | Return the output and specify its semantics. |

## D.5 Pseudocode and Code

The table below gives a package-light pseudocode layout; replace its contents with the actual method and keep correctness premises in the surrounding text.

**Listing D.1:** Code-listing format example.

```
// Use only short excerpts that help explain a design.
// Keep full implementations in the reproducibility artifact.
void search();
```